TASKFLOW

TASK MANAGEMENT WEB APPLICATION

MINOR PROJECT REPORT

Student Name	Nikhat Siddekee
Registered Email Address	nikhatsiddekee3@gmail.com
Program	Full Stack Web Development / MERN Stack Training Program
Batch	September 2026
Roll No.	WD-FSWD-A4/SEP-12005
Course	B.Tech in Computer Science and Engineering
Project Guide	Pratham Saxena
Academic Year	2026
Report Date	9 November 2026

Developed using HTML5, CSS3, JavaScript and Browser Local Storage

GitHub: github.com/Nikhat-codes/TaskFlow-Minor-Project

Live Demo: nikhat-codes.github.io/TaskFlow-Minor-Project/

CERTIFICATE

This is to certify that **Nikhat Siddekee**, a student of B.Tech in Computer Science and Engineering, has completed the minor project titled **“TaskFlow: Task Management Web Application”** under the guidance of **Pratham Saxena**. This report presents the project objectives, implementation, features and testing carried out for the application.

Project Guide: Pratham Saxena

Student: Nikhat Siddekee

Date: 9 November 2026

ACKNOWLEDGEMENT

I would like to thank my project guide, Pratham Saxena, for the guidance and feedback provided during this project. I am also thankful to everyone who supported me while planning, developing and documenting TaskFlow. Working on this project helped me practise the basics of front-end development and understand how HTML, CSS and JavaScript work together in a browser.

ABSTRACT

TaskFlow is a browser-based task management application developed using HTML, CSS and JavaScript. It provides a simple interface to create tasks, add descriptions, choose priorities and set due dates. Users can edit or delete tasks, mark them as completed or pending, search by task details, apply filters and sort the task list. A dashboard displays total, pending, completed and overdue task counts. Task information is saved in Browser Local Storage, so it remains available after refreshing the page in the same browser profile.

Keywords: Task management, HTML, CSS, JavaScript, Local Storage, responsive design.

1. INTRODUCTION

Managing assignments, personal work and deadlines can become difficult when tasks are written in different places or are not reviewed regularly. A task manager can make this process easier by keeping task details, priorities and due dates together. TaskFlow was developed as a small browser-based application to demonstrate these everyday task-management operations in one interface.

1.1 Problem Statement

When tasks are not organized, users may overlook deadlines or find it difficult to decide what needs attention first. TaskFlow addresses this by showing the task title, description, priority, due date and completion status together. Search, filters and sorting help users locate relevant tasks.

1.2 Project Objectives

• Build a simple and responsive task-management interface using HTML, CSS and JavaScript.

• Allow users to add, view, edit and delete tasks.

• Provide priority selection and due-date tracking.

• Support task completion, search, filtering and sorting.

• Show a dashboard summary and identify overdue pending tasks.

• Store task data in the browser so it can be restored after a page refresh.

1.3 Project Scope

The current version focuses on client-side task management in a web browser. It uses Browser Local Storage and does not require a separate server or database. Data is associated with the browser profile and device being used. User accounts, cloud backup and automatic synchronization between devices are outside the scope of this version.

1.4 Intended Users

The application is suitable for an individual who wants to maintain a personal list of assignments, study activities, household work or other everyday tasks. The current implementation is designed for single-user use in one browser profile.

2. TECHNOLOGY STACK

Technology	Purpose in TaskFlow
HTML5	Defines the page structure, dashboard, task form and task-list sections.
CSS3	Controls colours, spacing, cards, buttons and responsive layouts.
JavaScript	Handles task operations, validation, search, filters, sorting and dashboard updates.
Browser Local Storage	Saves the task array in the current browser profile and restores it after refresh.
Git and GitHub	Used for version control and hosting the project source code and documentation.

3. REQUIREMENTS

3.1 Functional Requirements

• Create a task with a title, description, priority and due date.

• Display saved tasks with their details and current status.

• Edit task details and delete a task after confirmation.

• Mark a task as completed or return it to pending.

• Search by task title or description.

• Filter tasks by completion status and priority.

• Sort tasks by creation time, due date or priority.

• Display total, pending, completed and overdue counts.

• Validate required fields and show a message when no tasks match.

• Save and restore tasks using Browser Local Storage.

3.2 Non-functional Requirements

• The interface should remain usable on desktop, tablet and mobile screens.

• The application should run in a modern browser without a separate backend server.

• Form controls and task actions should have clear labels.

• Task data should remain available after refreshing the same browser profile, subject to browser storage availability.

4. APPLICATION DESIGN

4.1 Page Structure

The page is divided into four main areas: a header with the application name, a dashboard with summary cards, a form for entering task details, and a task section containing search, filters, sorting and task cards. This arrangement keeps task creation and task review on the same page.

4.2 Task Data Structure

Each task is represented as a JavaScript object. The application stores the objects together in an array. The fields used by the current implementation are shown below.

Field	Purpose
id	Identifies a task; generated using the current timestamp.
title	Stores the task title.
description	Stores additional task details.
priority	Stores Low, Medium or High.
dueDate	Stores the selected due date.
createdAt	Stores the task creation date and time.
completed	Boolean value indicating whether the task is completed.

4.3 Application Workflow

1. The user opens **index.html** in a browser. 2. JavaScript reads saved task data from Local Storage. 3. The user fills in the task form and submits it. 4. The application validates the entered details and creates or updates the task object. 5. The task list and dashboard are refreshed. 6. The updated task array is saved to Local Storage. 7. On the next page load, the saved tasks are read and displayed again.

5. IMPLEMENTATION DETAILS

5.1 HTML Structure

The HTML file defines the main sections and form controls. IDs such as **taskForm**, **title**, **description**, **priority**, **dueDate**, **searchTask**, **statusFilter**, **priorityFilter**, **sortTasks** and **taskList** connect the page elements to the JavaScript code.

5.2 CSS and Responsive Layout

The stylesheet uses CSS variables for common colours and shadow values. Grid is used for the dashboard and filter controls, while Flexbox is used for task action buttons. Media queries adjust the layout at tablet and mobile screen widths. Priority levels have different coloured borders, completed tasks have a muted appearance, and overdue tasks receive a red highlight.

5.3 JavaScript Functions

Function / section	Role in the application
loadTasks()	Reads and parses saved tasks from Local Storage; returns an empty array if data cannot be read.
saveTasks()	Converts the task array to JSON and saves it to Local Storage.
getToday()	Creates today's date in YYYY-MM-DD format for overdue comparisons.
createElement()	Creates DOM elements and safely inserts text using textContent.
Form submit handler	Validates inputs and adds a new task or updates an existing task.
displayTasks()	Builds task cards and applies search, filters and sorting.
updateDashboard()	Calculates and displays total, pending, completed and overdue counts.

5.4 Validation and Error Handling

The form checks that the title, description and due date are provided. It also checks that the selected priority is one of the available values. The JavaScript prevents the form's default page submission and displays a message if required information is missing. Local Storage read and write operations are wrapped in error handling so the application can respond if browser storage is unavailable.

5.5 Search, Filters and Sorting

Search compares the entered text with the task title and description without depending on letter case. The status filter can show all, pending or completed tasks. The priority filter can show all tasks or one priority level. Sorting supports newest first, due date and priority; the priority order is High, Medium and Low.

5.6 Overdue Task Logic

A task is treated as overdue when it is still pending, has a due date, and its due date is earlier than the current date. Completed tasks are not counted as overdue. The dashboard count and task-card highlight help make these items noticeable.

6. USER INTERFACE

The screenshots below show the main parts of the application. They are included to connect the written description with the actual interface.

6.1 Main Dashboard

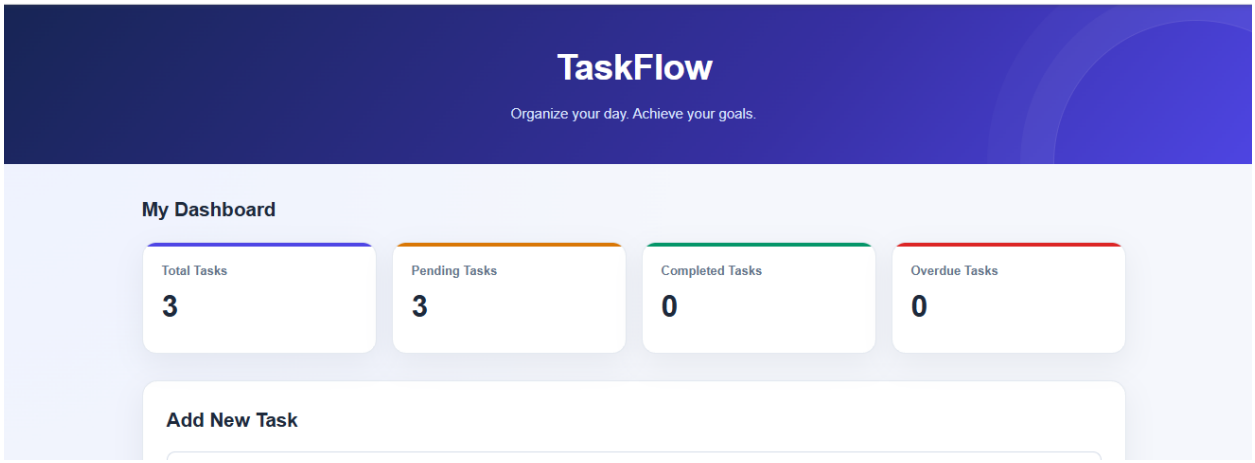


Figure 1. TaskFlow header and dashboard summary cards.

6.2 Task Creation Form

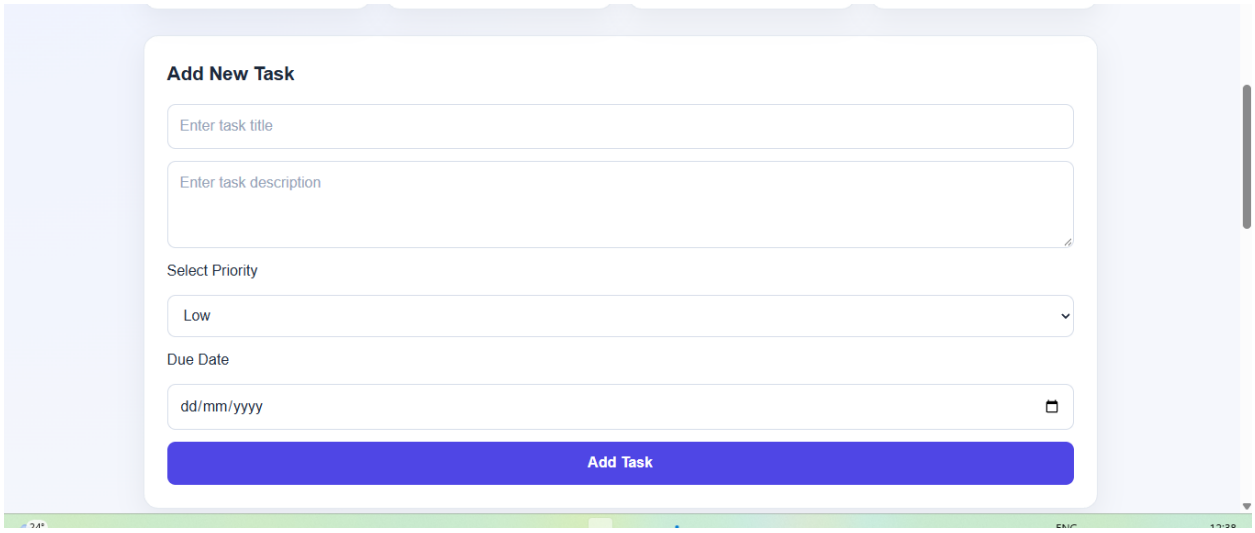


Figure 2. Form fields for task title, description, priority and due date.

6.3 Task List, Search, Filters and Sorting

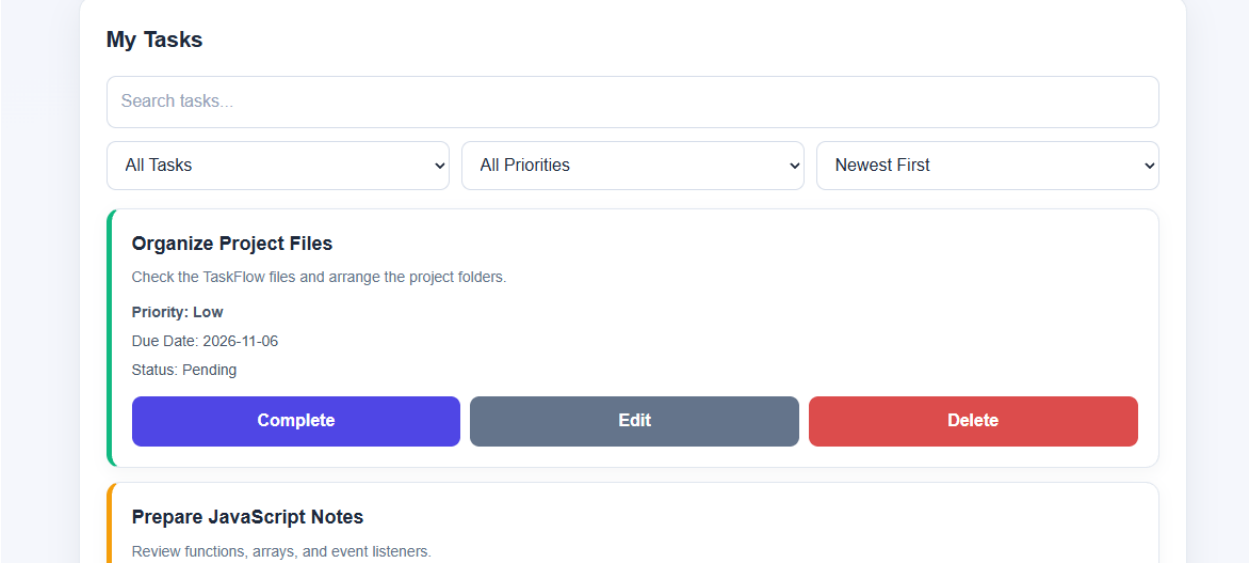


Figure 3. Task cards with search, filter, sorting and task action controls.

6.4 Responsive Mobile View

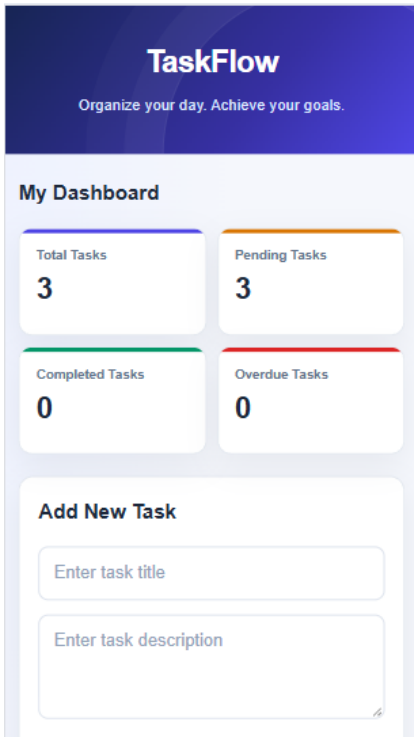


Figure 4. Dashboard and form displayed in a narrow mobile layout.

7. TESTING AND RESULTS

The following checks were recorded for the implemented features. The results indicate that the listed scenarios worked as expected during project testing.

ID	Test case	Expected result	Result
TC-01	Add task with valid details	Task appears and total count updates.	Pass
TC-02	Submit with required details missing	Task is not added.	Pass
TC-03	Edit an existing task	Updated details appear in the list.	Pass
TC-04	Delete a task	Confirmation is requested before deletion.	Pass
TC-05	Complete or reopen a task	Status and dashboard counts update.	Pass
TC-06	Search by task details	Matching tasks are displayed.	Pass
TC-07	Filter by status or priority	Only matching tasks are displayed.	Pass
TC-08	Change sorting option	Tasks appear in selected order.	Pass
TC-09	Refresh the page	Previously saved tasks are restored.	Pass
TC-10	Check an overdue pending task	Task is highlighted and counted as overdue.	Pass
TC-11	View on a mobile-sized screen	Layout fits a narrow screen.	Pass

7.1 Testing Notes

The checks cover the main user actions: creating and editing tasks, deleting with confirmation, changing completion status, searching, filtering, sorting, restoring data after refresh and viewing the interface at a mobile-sized width. These are functional checks of the current browser-based implementation.

8. CHALLENGES AND SOLUTIONS

Challenge	Approach used
Keeping tasks after refresh	Used Browser Local Storage to save the task array and load it when the page opens.
Showing task progress clearly	Added dashboard counts for total, pending, completed and overdue tasks.
Finding a particular task	Added text search, status filtering, priority filtering and sorting controls.
Identifying missed deadlines	Compared pending task due dates with the current date and added an overdue label.
Supporting smaller screens	Used CSS Grid, Flexbox and media queries to adapt the layout.

9. LIMITATIONS AND FUTURE ENHANCEMENTS

The current version stores task data only in the browser's Local Storage. It does not include user accounts, cloud synchronization, shared task lists or automatic reminders. If browser data is cleared or the user changes browser profiles or devices, the saved task list may not be available there.

Possible future improvements include:

• Optional user login and cloud-based task storage.

• Task categories, labels and recurring tasks.

• Calendar view for reviewing upcoming due dates.

• Reminders and browser notifications.

• Export and import options for task data.

10. CONCLUSION

TaskFlow is a simple task management web application built with HTML5, CSS3 and JavaScript. It brings task creation, editing, deletion, completion tracking, search, filtering, sorting and overdue indicators into one responsive page. Browser Local Storage allows the task list to be restored after a refresh in the same browser profile. Developing this project provided practical experience in structuring a web page, styling responsive layouts, handling DOM events, working with arrays and objects, validating form input and using browser storage.

11. PROJECT LINKS

GitHub Repository: https://github.com/Nikhat-codes/TaskFlow-Minor-Project

Live Demo: https://nikhat-codes.github.io/TaskFlow-Minor-Project/

12. REFERENCES

• MDN Web Docs — HTML: https://developer.mozilla.org/docs/Web/HTML

• MDN Web Docs — CSS: https://developer.mozilla.org/docs/Web/CSS

• MDN Web Docs — JavaScript: https://developer.mozilla.org/docs/Web/JavaScript

• MDN Web Docs — Web Storage API: https://developer.mozilla.org/docs/Web/API/Web_Storage_API

• TaskFlow project source files: index.html, style.css and script.js.

Project: TaskFlow — Task Management Web Application
Student: Nikhat Siddekee
Course: B.Tech in Computer Science and Engineering
Project Guide: Pratham Saxena